

1(a) A program is designed to use a stack data structure.

The current contents of part of the stack and its current pointer value are shown:

5	
4	
3	
2	"l"
1	"n"
0	"k"

pointer = 2

Show the contents of the stack and its pointer value after the data items "t" and "s" are inserted in the order given.

5	
4	
3	
2	
1	
0	

pointer =

[2]

(b) The stack is stored using a 0-based 1-dimensional array called theStack with 20 elements.

The function push() stores its parameter into the next space in the stack if it is not full. The function returns true if it is successfully stored and returns false if the stack is full.

Complete the function push() using pseudocode or program code.

```
function push(data)

    if pointer == ..... then

        return false

    else

        pointer = .....

        theStack[pointer] = .....

        .....

    endif

endfunction
```

[4]

(c) A pointer value of -1 indicates that the stack is empty.

The function `pop()` returns the next element in the stack. If the stack is empty it returns "false". The function updates the pointer value when appropriate.

Write the function `pop()` using pseudocode or program code.

[6]

- (d) A program is designed to reverse a set of data using the stack.

The main program takes 10 characters separately as input from the user and uses the function `push()` to insert these into the stack.

Once all 10 characters have been inserted into the stack, the program then uses the function `pop( )` to output all 10 characters from the stack.

Write the main program to achieve this using pseudocode or program code.

[5]

- 2(a) Two data structures are a queue and a stack.

Describe **one** difference between a queue and a stack.

[2]

- (b) The current contents of a stack created from an array called `stack` and pointer called `topPointer` are shown:

index	stack
7	
6	

5	
4	
3	
2	13
1	21
0	10

The topPointer currently stores 2.

- i. State the purpose of topPointer in the stack.

----- [1]

- ii. Describe the steps that will be followed to remove an item from the stack.

----- [3]

- iii. Complete the function to insert a new data item onto the stack.

```
function push(dataItem)
    if topPointer ..... 7 then
        return false
    else
        topPointer = .....
        stack[topPointer] = .....
        return true
    endif
endfunction
```

[3]

3(a) A programmer is designing a program that will store data.

The programmer is deciding whether to store the data in a stack or a queue.

Identify **one** similarity and **one** difference between a stack and a queue.

Similarity

.....

Difference

..... [2]

(b) The pseudocode function, enqueue, inserts an item into a queue.

```
01 function enqueue(item)
02     if tailPointer >= queue.length then
03         return false
04     else
05         queue[tailPointer] = item
06         tailPointer = tailPointer + 1
07     return true
08 endif
09 endfunction
```

i. Give the name of the parameter in the function enqueue.

..... [1]

ii. Give the name of **one** global variable that is used in the function enqueue.

..... [1]

iii. Describe **one** benefit and **one** drawback of using global variables instead of parameter passing in a subroutine.

Benefit

.....

Drawback

..... [4]

- iv. The function enqueue can be called by the main program.

Explain why the function enqueue returns true or false values, and how this can be used by the main program that calls the function.

[3]

- v. The pseudocode function, dequeue, removes and returns the first item in the queue. If the queue is empty, the function returns the string "EMPTY".

```
01 function dequeue(data)
02     if headPointer != tailPointer then
03         return "EMPTY"
04     elseif
05         value = queue[headPointer]
06         return value
07         headPointer = headPointer + 1
08     endif
09 endfunction
```

The function dequeue has **several** errors.

Identify the line number of any **three** errors **and** state the correction required.

Error 1 Line Number

Error 1 Correction

Error 2 Line Number

Error 2 Correction

Error 3 Line Number

Error 3 Correction [3]

vi. The programmer has corrected all of the errors in the function dequeue.

The main program repeatedly calls the function dequeue until all of the elements in the queue have been output.

Write the main program using pseudocode or program code.

[3]

(b) enqueue will add data to the queue. dequeue will remove data from the queue.

Show the contents of the queue and the position of both pointers after the following actions have been executed on the queue shown in Fig. 4 in the order given:

- enqueue(20)
- dequeue()
- dequeue()

--	--	--	--	--	--	--	--

[2]

(c) The queue is used to store ID numbers of jobs that a program needs to process. Some jobs will be given a priority which means they need to be processed first.

Explain why this queue is **not** a suitable data structure for this program.

[2]

6 Describe how an array can be used to store and access data in a stack data structure.

[4]

END OF QUESTION PAPER

Question			Answer/Indicative content	Marks	Guidance						
1	a		1 mark for stack contents 1 mark for pointer value <table><tr><td></td></tr><tr><td>"s"</td></tr><tr><td>"t"</td></tr><tr><td>"l"</td></tr><tr><td>"n"</td></tr><tr><td>"k"</td></tr></table> pointer = 4		"s"	"t"	"l"	"n"	"k"	2	Allow the absence of quotation marks around the values in the stack Allow l or L if difficult to determine letter/case etc. Examiner's Comments Most candidates successfully demonstrated the ability to add new values to the stack in the correct order.
"s"											
"t"											
"l"											
"n"											
"k"											
	b		1 mark for each completed statement to max 4 function push(data) if pointer == 19 then return false else pointer = pointer + 1 theStack[pointer] = data return true endif endfunction	4	Allow len(theStack) - 1 or theStack.length - 1 as equivalent to 19 Examiner's Comments Most candidates made a successful attempt at this question. But many did not achieve full marks. A common misconception was to compare the pointer with 20 instead of 19, showing limited understanding of zero-indexing and the purpose of the pointer in this code being to point to the last item inserted into the stack.						
	c		1 mark each to max 6 • Function pop() declaration • Checking if stack is empty ... • ...and returning "false" • (otherwise) accessing the data at pointer • Decrementing pointer before returning • Returning the data accessed e.g. function pop() if pointer == -1 return "false"	6	BP1 ignore any parameters Allow equivalent check in BP2 e.g. if pointer < 0 BP3 is strictly dependent on MP2 being met BP3 allow Boolean false instead of string "false" Allow BP4 and BP5 to be in the opposite order with appropriate logic. BP4 and BP6 can be combined Allow reversal of logic and other						

Question	Answer/Indicative content	Marks	Guidance
	<pre> else dataToReturn = theStack[pointer] pointer = pointer - 1 return dataToReturn endif endfunction </pre>		equivalent solutions e.g. <pre> function pop() if pointer > -1 pointer = pointer - 1 return theStack[pointer+1] else return "false" endif endfunction </pre> Examiner's Comments Many candidates made a good attempt to write a pop () function with many candidates being given 5 or 6 marks. A common misconception for this question was to return the value of the stack at the pointer without sequencing the pointer decrement correctly. Exemplar 1 <pre> def pop(): if pointer == -1: return False else: return theStack[pointer] Value = theStack[pointer] return Value pointer = pointer - 1 </pre> This response was given 5 marks and exemplified a common misconception. Many candidates returned the value at the top of the stack and then decremented the pointer. This would mean that the line of code to decrement the pointer would not run. The pointer decrement needed to be placed before the return value line.

Question		Answer/Indicative content	Marks	Guidance
	d	1 mark each to max 5 • Loop 10 times... • ...to take inputs... • ...calling push() with each value input • Calling pop() for each pushed value... • ...and outputting each value in the correct order e.g. <pre>for count = 0 to 9 push(input("Enter a string")) next count</pre> <pre>for count = 0 to 9 print(pop()) next count</pre>	5	Award follow through if incorrect number of items input/output Loop constructs must be valid for MP2 i.e. a fully working for or while loop or equivalent. If Python syntax is used it must be clear that a for loop executes 10 times e.g. for i in range(0,10) Examiner's Comments Many candidates were less successful in producing pseudocode to an open-ended problem. Many candidates did not create accurate loop structures that would iterate 10 times. There were few completely successful responses. Less successful responses often did not output values popped from the stack or iterated an incorrect number of times. Exemplar 2 Once all 10 characters have been inserted into the stack, the program then uses the function pop() to output all 10 characters from the stack. Write the main program to achieve this using pseudocode or program code. <i>for loop input push</i> <i>for loop pop 2 correct</i> <pre> For x = 0 to 9 y = 0 ans = "" For x in range 0 to 9: data = input("Enter character: ") push(data) For x = x + 1 For y in range 0 to 9: not pop ans = ans + pop() y = y + 1 print(ans) </pre> [5] This response gained 4 marks. The candidate wrote a mix of Python and pseudocode. This response shows how the variable has been incremented inside of the loop. Where candidates produced an incorrect loop, follow through was applied and marks were gained for the subsequent input and push operations. This solution appended the popped values before outputting to produce a valid solution. Again, the output push loop was incorrect, but it operated the same number of times as the input push loop, so follow through was applied.
		Total	17	

Question			Answer/Indicative content	Marks	Guidance
2	a		1 mark each to max 2 e.g. - Queue is FIFO - Stack is LIFO - Queue inserts data and removes data at different ends - Stack inserts and removes at the same end - Queues need two pointer values - Stacks (usually) need one pointer value - Queue can be circular - Stack can only be linear	2	Must be full comparison for 2 marks. Examiner's Comments Nearly all candidates identified that a stack is a LIFO structure whereas a queue is a FIFO structure. Few of the other potential responses on the mark scheme were seen.
	b	i	1 mark for: - Points to the index of the last value put into the stack // points to the item at the top of the stack - Points to the next item that will be output from the stack	1	Any equivalent description Examiner's Comments While many candidates identified that topPointer was a pointer to the index where that last item was inserted into the stack, some candidates confused this by saying that it actually held the last value, implying or stating that it held the value 13.  Misconception Some candidates think that a pointer holds the actual value that it points towards, rather than holding the location that contains the value.
		ii	1 mark each to max 3: e.g. - Check if stack is empty // check if TopPointer == -1/null - If not, then take/remove/pop data at index TopPointer - Decrement TopPointer	3	Allow decrement topPointer followed by return stack[topPointer+1] Examiner's Comments Many candidates achieved some of the marks, but far fewer were given full marks. The most omitted part of the required response was checking whether the stack was empty or not before trying to pop the value at the top of the stack.

Question			Answer/Indicative content	Marks	Guidance
		iii	1 mark for each answer <pre> function push(dataItem) if topPointer == 7 then return false else topPointer = topPointer + 1 stack[topPointer] = dataItem return true endif endfunction </pre>	3	Allow = instead of == Accept >= instead of == <u>Examiner's Comments</u> Many candidates produced accurate solutions. The most common error was an incorrect relational operator in the first gap with <= often given as an incorrect response instead of <.
			Total	9	

Question			Answer/Indicative content	Marks	Guidance
3	a		1 mark for similarity: - Both are data structures / store data - Both allow data to be added/removed - Both have identifier(s) / pointer values 1 mark for difference: - Stack is LIFO and queue is FIFO - Queue supports enqueue/dequeue operations whilst stack supports push/pop operations - Queue require two pointers whilst stacks only require one pointer	2	<u>Examiner's Comments</u> Most candidates demonstrated knowledge of the purpose of stacks and queues. For the similarity, a frequent response was that both stacks and queues are data structures used to store data. For the difference, most candidates identified that a stack was a LIFO structure while a queue was a FIFO structure.
	b	i	item	1	Allow queue can be circular or linear, stack can only be linear <u>Examiner's Comments</u> The vast majority of candidates identified that item was the paramete
		ii	tailPointer	1	Allow queue <u>Examiner's Comments</u> Many candidates identified that tailPointer was an example of a global variable in the code.

Question			Answer/Indicative content	Marks	Guidance
		iii	1 mark for benefit and 1 mark for expansion: e.g. • Simpler to program ... • ... because values do not need to be passed/renamed/moved between different subroutines • Do not need to worry about returning values // do not need to decide between byval/byref ... • ... all parts of the program can access the (same) value (in the same way) 1 mark for drawback and 1 mark for expansion: • Uses more memory ... • ... because the memory space is declared when the program starts and remains in use throughout • Makes testing / debugging more difficult ... • as it's difficult to test an individual block of code • Reduces data accuracy / integrity ... • changing a global variable may have an impact on another module	4	2 marks max for benefit 2 marks max for drawback <u>Examiner's Comments</u> The majority of candidates struggled to describe the implications of using global variables, and this highlighted a lack of appreciation of parameter passing and variable scope. At AS Level candidates should become more adept at using parameters and develop an understanding of why this is beneficial – both in theoretical terms and within their personal programming practice. There were few responses that were able to give complete descriptions.

Question			Answer/Indicative content	Marks	Guidance
		iv	1 mark per bullet to max 3 - False is returned if the queue is full the main will not attempt to add another item - True is returned if the item is successfully added so the main program can try to add another item	3	Examiner's Comments Some candidates thought that if enqueue() returned True it meant that there would be space to add additional values. This was not the case; it meant that the item had been enqueued successfully, but it could have been the last item that then filled the queue to capacity. Sometimes candidates did explain the reasons why True and False would be returned but were then unable to go on to say how they could be used. Good responses were seen such as 'the main program can give an error message if the operation didn't work if the queue was full'. Exemplar 1 The enqueue() function returns "False" in the case that the queue is full (ie the tail pointer is greater than or equal to the predetermined length) and the return indicates the item has not been added. The function returns "True" if the queue is not full and indicates the item has been added. These return values can be used to check whether the item passed to enqueue() has been successfully added by assigning them to a variable: successful = enqueue("A") for example. [3] This response showed clear understanding of the need to store the result returned by dequeue() to be used in the conditional loop test.
		v	1 mark for error and correction to max 3 Line 01 There is no need to pass in a parameter Line 02 has != instead of == Line 04 has elseif and no condition // replace with else Line 06 should be headpointer = headpointer + 1 // Swap lines 06 and 07 Line 07 should be return value	3	Do not award marks for the line number. The error must be related to the line number stated. Examiner's Comments Many candidates identified at least one logical error, but few were able to identify three errors. The most common error identified was the error in line 02.

Question			Answer/Indicative content	Marks	Guidance
		vi	1 mark per bullet to max 3 • Use of loop to call dequeue().... • ...to output the return value... • ...until "EMPTY" is returned e.g. <pre> elementsLeft = true while elementsLeft == true value = dequeue() if not(value = "EMPTY") then print(value) else elementsLeft = false endif endwhile </pre>	3	<u>Examiner's Comments</u> Some candidates mistakenly gave a definition for the dequeue() function instead of answering the question set. Candidates need to be mindful to read the question carefully. Some candidates used a count-controlled loop whereas a conditional loop was required. Those who made a good attempt at a response often did not assign the result of dequeue () to a variable, so could not effectively use the return value from the function as well as print the value returned.
			Total	17	

Question		Answer/Indicative content	Marks	Guidance
4	a	1 mark each • Check if the stack is empty // check topStack is equal to 0 • ...and if so return a suitable value (e.g. -1/ null) // do nothing //give warning • (If not empty) decrement topStack • Return the value in element topStack from the array numbers	4	Do not award BP3 if a value has been returned from the function for BP4 first. Examiner's Comments There were many weak responses that did not outline the discrete steps required in the pop () function and there was often no reference to the topStack pointer. A number of candidates confused a stack with a queue and talked about head/rear pointers. Some candidates confused 'space for 100 elements' in the question with the top of the stack and then erroneously talked about removing item 100. Some candidates erroneously stated that the value at topStack - 1 would be returned before saying that topStack would be decremented. The order of the steps was important for the function to operate correctly, and many candidates lost a mark due to this. Exemplar 2 Describe the steps in the function pop (). <i>if pointer is -1</i> <i>1. checks if stack is empty, if yes returns an error or nothing to return</i> <i>2. goes to topStack - 1 value and return the value and deletes it from the pointer</i> <i>3. moves pointer at topStack to topStack - 1</i> <i>=> topStack = topStack - 1</i> This response shows the candidate returning the value from topStack - 1 in step 2. As soon as the function returns a value no further actions are performed within the function, so decrementing topStack in point 3 was not given.
	b	1 mark for each completed statement <pre> function push (dataValue) if topStack != 100 then numbers[topStack] = dataValue topStack = topStack + 1 return true else return false endif end function </pre>	4	Examiner's Comments The majority of candidates scored three or more marks. Some candidates erroneously used numbers.length or len(numbers) in the second space instead of topStack.

Question		Answer/Indicative content	Marks	Guidance
	c	1 mark each • Calling push() with parameter 15 • ...storing/using return value in selection ... • ...comparing true/false (may be implicit e.g. if push(15) then) ... • ...outputting a suitable message if false and if true e.g. <pre> added = push(15) if added = false then print("Not added") else print("Added") endif </pre> <pre> if push(15) then print("Added") else print("Not Added") endif </pre>	4	True/False comparisons must be Boolean values and not strings, but allow FT after that. If push () is called twice BP4 cannot be awarded. <u>Examiner's Comments</u> Some candidates erroneously tried to make a call such as Function Push (15) instead of calling and using/storing the result of Push (15). A number of responses incorrectly used string values "True" / "False" instead of Boolean True / False. Many candidates tried to directly access and use the topStack pointer instead of using the function return value as required in the question.
		Total	12	

Question		Answer/Indicative content	Marks	Guidance
5	a	1 mark each - headPointer to identify the first item/element in the queue // identify which item to dequeue/remove next - tailpointer to identify the next free space in the queue // identify where the next item/element will be enqueued/added	2	<u>Examiner's Comments</u> This question was generally well answered by many candidates. There were some vague responses such as 'start of queue', which was not precise enough to indicate the first item, or the index of the first item, as distinct to the indexes of the actual data structure itself.
	b	1 mark for queue elements 1 mark for both pointers 	2	Allow 20 and 15 in place but crossed out OR allow 20 and 15 in place only if headPointer and tailPointer are correct <u>Examiner's Comments</u> Most candidates either tended to score full marks or no marks. Many candidates missed labelling the pointers in their diagrams and just gave updated queue values. Some candidates kept the values 20 and 15 on the diagram and correctly moved the headPointer to point to index 3 and the tailPointer to the next free space available. A considerable number of candidates erroneously shifted all items forward, showing a lack of understanding as to how a queue is efficiently implemented.
	c	1 mark each to max 2 e.g. - A queue is a FIFO structure // elements processed in the order entered - A queue will not allow new data inserted at the front // only allows new data to be enqueued at the rear - The queue contents cannot be resequenced/sorted without rewriting	2	<u>Examiner's Comments</u> The first part of many responses was well attempted, with most candidates identifying the First In First Out (FIFO) property of a queue. Far fewer candidates were able to successfully expand on this for the second mark. A linked explanation was required such as a higher priority item cannot be inserted at the front/in the middle of the queue because only the first element can be accessed/dequeued. Some candidates just reiterated what FIFO meant instead of giving the linked explanation to a priority queue which was insufficient to gain the second mark.
		Total	6	

Question	Answer/Indicative content	Marks	Guidance
6	1 mark per bullet to max 4 • (Stack) Pointer points to the last element added to the stack / top of the stack • New data is added to the pointer position // pointer+1... • ...check for overflow condition • ...pointer is then incremented • Data is removed from pointer/pointer-1 position... • ...check for underflow condition • ...pointer is then decremented • Elements can be accessed through Push() and Pop() methods that are implemented	4	Note: Answers must relate to an array implementation which means that a stack pointer must be implemented. Examiner's Comments Candidates need to be mindful of the fact that an array is a static data structure with a predetermined size. This meant that responses that explained how lists could be used to append and remove items were not given marks. This is an area where candidates who only have programming experience of using lists in Python often struggle. Implementing a stack in an array requires a stack pointer. Those candidates who appreciated this and who clearly had practical experience of modelling push()/pop() operations on a stack were able to articulate how the stack would access array locations at the stack pointer. Exemplar 3 <i>you can create a one dimensional array and assign two pointer index values pointing to the first item as the bottom and then another pointing to the last as the top. if an item is popped the item with the index position of the top is deleted and the top pointer's value decreased by one. if you want to add an item you append to the list to the position of the top. the top item you should check if top pointer = bottom pointer, as stack is empty, and if (top+1) is greater than the length of the array to see if the stack is full.</i> This exemplar was one of the few responses to show a clear insight as to how an array can be used to store and access a stack. It makes it clear that a stack pointer is required and goes on to explain how the stack pointer is manipulated when push()/pop() operations are performed.  Misconception Many candidates were not clear about the difference between an array and a list: • An array is a static structure whose

Question			Answer/Indicative content	Marks	Guidance
					size cannot be changed. • Lists are dynamic and support items being removed or appended.
			Total	4	